DATABASE SYSTEMS · WEEK 2

Relational Model, ER Modeling & Normalization

From a messy spreadsheet to a schema that cannot lie to you

Keys · ERD · ERD→relations · 1NF–BCNF · hands-on practice



ROADMAP

What we do today

1. **Relational model & keys**

   The vocabulary you will use all semester: relation, tuple, domain, superkey, candidate key, primary key, foreign key.

2. **ER modeling fundamentals**

   Entities, attributes, relationships, cardinality, weak entities — design before you type CREATE TABLE.

3. **Mapping ERD → relations**

   Seven mechanical rules that turn any diagram into tables with the right keys.

4. **1NF, 2NF, 3NF, BCNF**

   What each normal form actually prevents — and the exact smell that gives away a violation.

5. **Practice: normalize a real table**

   One ugly delivery log → six clean tables, step by step, with you at the whiteboard.

> Running example all day: a food-delivery service — customers, restaurants, menu items, orders, couriers.

1

Relational model & keys

The five words you must never mix up — and the three kinds of key.

VOCABULARY

A relation is a set of tuples

customer_id	full_name	city	phone
1001	Aigul S.	Almaty	+7 701 111 22 33
1002	Daniyar K.	Astana	+7 707 444 55 66
1003	Marat T.	Almaty	+7 775 888 99 00

relation customer

- **Relation** — the table itself — customer
- **Attribute** — a column — city
- **Tuple** — a row — (1002, Daniyar K., Astana, …)
- **Domain** — the set of values an attribute may take
- **Degree** — number of attributes — here 4
- **Cardinality** — number of tuples — here 3

A relation is a SET: no row order, no duplicate rows. SQL relaxes this — SELECT returns a multiset, and without ORDER BY the order is undefined.

ATOMICITY & NULL

Two rules that are broken every semester

RULE 1 — VALUES MUST BE ATOMIC

```
phone = '+7 701 111 22 33, +7 707 444 55 66'
```

- "Find everyone with a Beeline number" becomes string parsing
- No index can help; every query is a full scan
- Fix: one value per cell → customer_phone(customer_id, phone)

RULE 2 — NULL IS NOT A VALUE

- NULL means "unknown" or "not applicable" — never 0, never ''
- salary > 100000 and salary <= 100000 together still miss NULL rows
- NULL = NULL is unknown → use IS NULL / IS DISTINCT FROM

THREE-VALUED LOGIC — COMPARISON WITH NULL

expression	result	row passes WHERE?
salary > 100000 (salary = 120000)	true	yes
salary > 100000 (salary = 50000)	false	no
salary > 100000 (salary IS NULL)	unknown	no — and it is not in the opposite result either

KEYS

Superkey → candidate key → primary key

`student ( student_id , iin , email , full_name , group_code )`

Superkey

Any attribute set whose values identify a row uniquely — extra columns allowed.

`{student_id}` `{iin}` `{email}`
`{student_id, full_name}` `{iin, group_code}`

Candidate key

A superkey that is minimal — remove any attribute and uniqueness is lost.

`{student_id}` `{iin}` `{email}`

Primary key

The one candidate key we choose to identify rows and to be referenced by others.

`student_id`

> A key is a claim about EVERY possible state of the table, not about the rows you happen to have today.

PRACTICAL CRITERIA

Which candidate key becomes the primary key?

Candidate	Unique	Never changes	Never NULL	Verdict
email	yes	no — people change it	yes	**avoid**
iin (national ID)	yes	yes	not for foreign students	**avoid — also sensitive data**
student_id (surrogate)	yes	yes	yes	**primary key**

NATURAL VS SURROGATE

- Natural key: already exists in the domain (iin, ISBN, IATA code)
- Surrogate key: invented by us (bigserial / identity / UUID)
- Default in practice: surrogate PK + UNIQUE constraint on the natural key
- That way you keep the business rule and still have a stable FK target

COMPOSITE KEYS ARE NORMAL

```
enrollment ( student_id, course_id, semester, grade )
PRIMARY KEY (student_id, course_id, semester)
```

- All three attributes are needed: a student may retake a course
- Drop semester and you cannot store the retake at all
- Composite PK is the standard shape of a junction table (see part 3)

REFERENTIAL INTEGRITY

Foreign keys: the only thing that keeps data honest

```
CREATE TABLE orders (
  order_id     bigserial PRIMARY KEY,
  customer_id  bigint NOT NULL
                 REFERENCES customer(customer_id)
                 ON DELETE RESTRICT,
  courier_id   bigint
                 REFERENCES courier(courier_id)
                 ON DELETE SET NULL,
  placed_at    timestamptz NOT NULL DEFAULT now()
);
```

A foreign key says: every value here must already exist there — or be NULL.

ON DELETE — CHOOSE PER RELATIONSHIP

RESTRICT / NO ACTION

Block the delete. Default choice for anything financial: an order must not lose its customer.

CASCADE

Delete the children too. Correct only when the child cannot exist alone: order → order_item.

SET NULL

Child survives, link is cleared. Fits optional links: order → courier.

Rule of thumb: CASCADE only for parts of a whole. If the child is a business fact in its own right, use RESTRICT.

5 MINUTES, IN PAIRS

Practice 1 — find the keys

```
flight_seat ( flight_no, flight_date, seat_no, passenger_iin, passenger_name, aircraft_model )
```

Business rules: ① one seat on one flight on one date is sold to at most one passenger · ② a passenger takes at most one flight per day

1. Write down two different superkeys.
2. Find ALL candidate keys. Prove minimality: remove one attribute and show a counter-example.
3. Which one would you make the primary key — and would you introduce a surrogate key instead? Why?
4. Which attribute in this table already looks like it belongs somewhere else?

> Answers are in the speaker notes of this slide — do not reveal before the pairs report back.

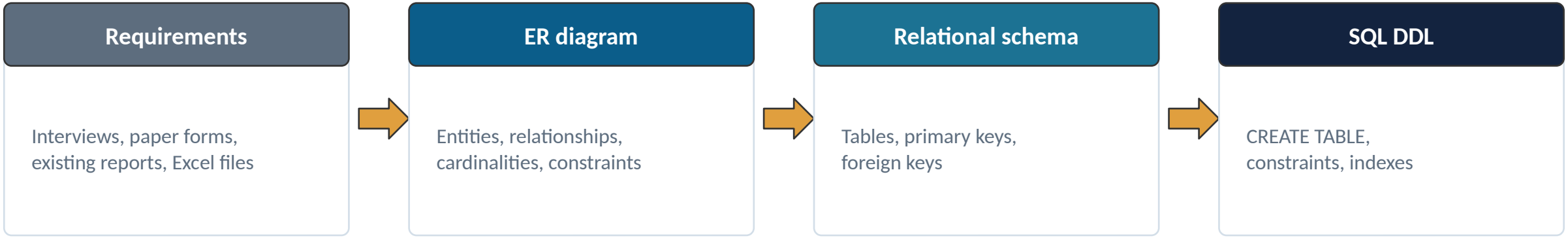
2

ER modeling fundamentals

Designing the picture before you write a single CREATE TABLE.

WHY BOTHER

Where the ERD sits in the process



WHAT AN ERD BUYS YOU

- A picture a non-programmer can review and correct
- Cardinalities force you to ask the awkward questions early
- Mapping to tables becomes mechanical, not creative

THE COST OF SKIPPING IT

- Fixing a wrong cardinality in the diagram: minutes
- Fixing it after 2 million rows are loaded: a migration window, a backfill script and a night shift
- Most "database performance problems" are design problems

ER NOTATION

The four building blocks



Entity

A thing we keep facts about.
Becomes a table.



Attribute

A property of an entity.
Becomes a column.



Relationship

A fact linking entities.
Becomes an FK or a table.

1 N

Cardinality

How many of each side
take part. Decides the shape.

HOW TO READ A RELATIONSHIP OUT LOUD — ALWAYS IN BOTH DIRECTIONS

```
"One customer places many orders."        →   1 : N
"One order is placed by exactly one customer."   →   confirms the N side
```

If the second sentence is false, it is not 1:N — it is M:N, and you need a third table.

ER NOTATION

Attribute types — and what each one costs you

Type	Example in our domain	What it means	How it lands in tables
Simple	placed_at	One indivisible value	One column
Composite	address → city, street, building	Made of meaningful parts	One column per part — never one text field
Multivalued	customer phones { … }	Many values for one entity	A separate table — this is a 1NF issue
Derived	order_total = Σ qty × price	Computable from other data	Do not store it unless you can prove you must
Key	customer_id	Identifies the entity	PRIMARY KEY

COMPOSITE: THE CLASSIC MISTAKE

address text → "Almaty, Tole bi 59, apt 12"
Now write: "all orders delivered in Almaty".

DERIVED: WHEN STORING IS RIGHT

Store order_total when the price may change later — the order must remember what it charged, not what the menu says today.

1:1 · 1:N · M:N

Cardinality decides the number of tables

	EXAMPLE	EFFECT ON THE SCHEMA	HOW
1 : 1	**Courier ↔ Vehicle currently assigned**	One extra column	`FK on one side + UNIQUE on it`
1 : N	**Restaurant → MenuItem**	Still two tables	`FK on the N side (menu_item.restaurant_id)`
M : N	**Order ↔ MenuItem**	A THIRD table appears	`order_item(order_id, item_id, …) with composite PK`

M:N is the only cardinality that creates a table you did not draw as an entity. Students forget it in every exam — check for it first.

THE CONSTRAINTS PEOPLE FORGET

Participation and weak entities

TOTAL VS PARTIAL PARTICIPATION

- Total (double line): every order MUST have a customer → customer_id NOT NULL
- Partial: a customer MAY have no orders → nothing to enforce
- Ask literally: "can one exist without the other?"
- This is where NOT NULL comes from — not from taste

WEAK ENTITY

- Has no identity of its own — only inside its owner
- order_item line #2 means nothing without order #4711
- PK = owner PK + partial key: (order_id, line_no)
- Almost always FK … ON DELETE CASCADE

STRONG OR WEAK? DECIDE WITH ONE QUESTION: DOES IT SURVIVE ON ITS OWN?

Customer	Order	OrderItem	Restaurant	MenuItem	DeliveryAttempt
strong	strong	weak	strong	strong	weak

REFERENCE CARD

ER notation — the complete legend



Entity

A thing we keep facts about

→ a table



Weak entity

No identity of its own

→ PK includes owner



Attribute

A property of an entity

→ a column



Key attribute

Identifies the entity

→ PRIMARY KEY



Multivalued

Many values for one entity

→ a child table (1NF)



Derived

Computable from other data

→ usually not stored



Relationship

A fact linking entities

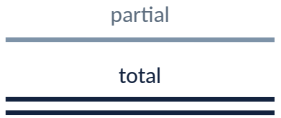
→ an FK or a table



Identifying rel.

Ties a weak entity to its owner

→ ON DELETE CASCADE



Participation

Single = partial, double = total

→ NULL vs NOT NULL

1 : N

M : N

Cardinality

How many on each side

→ where the FK goes

Chen notation is what we draw on the board. Every symbol on this page has exactly one job: to decide something in the SQL schema.

WHAT TOOLS ACTUALLY DRAW

The same thing in crow's foot notation



Exactly one

Mandatory, single. NOT NULL FK



Zero or one

Optional, single. Nullable FK



One or many

Mandatory, multiple



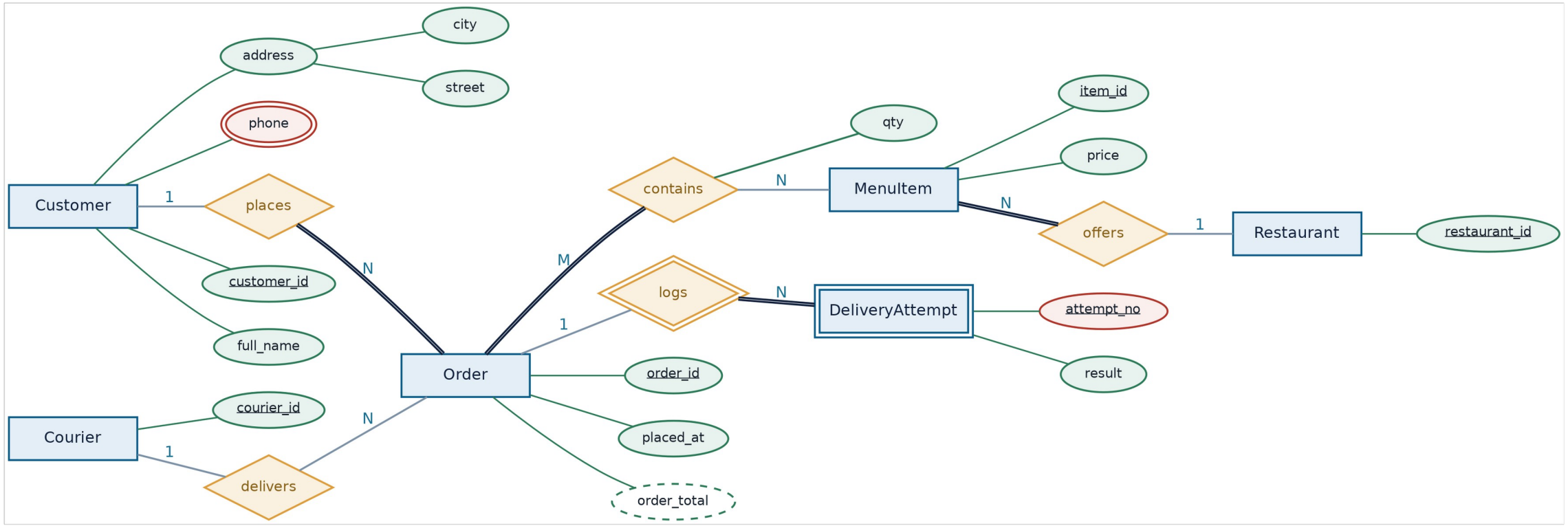
Zero or many

Optional, multiple. The default case

Chen	Crow's foot	Reads as	In SQL
1 : N	one-and-only-one ↔ zero-or-many	One customer has many orders	FK on the many side
M : N	zero-or-many on both ends	An item appears in many orders	Junction table, composite PK
1 : 1	one-and-only-one on both ends	One order has one invoice	FK + UNIQUE
Double line	tick nearest the entity	Every order must have a customer	NOT NULL
Double rectangle	Solid line (identifying)	A line item lives inside its order	PK includes parent + CASCADE
—	Dashed line (non-identifying)	A courier exists without any order	Plain FK

The mark sits at the FAR end of the line: the symbol next to Order says how many Orders, not how many Customers. This is the most common misreading.

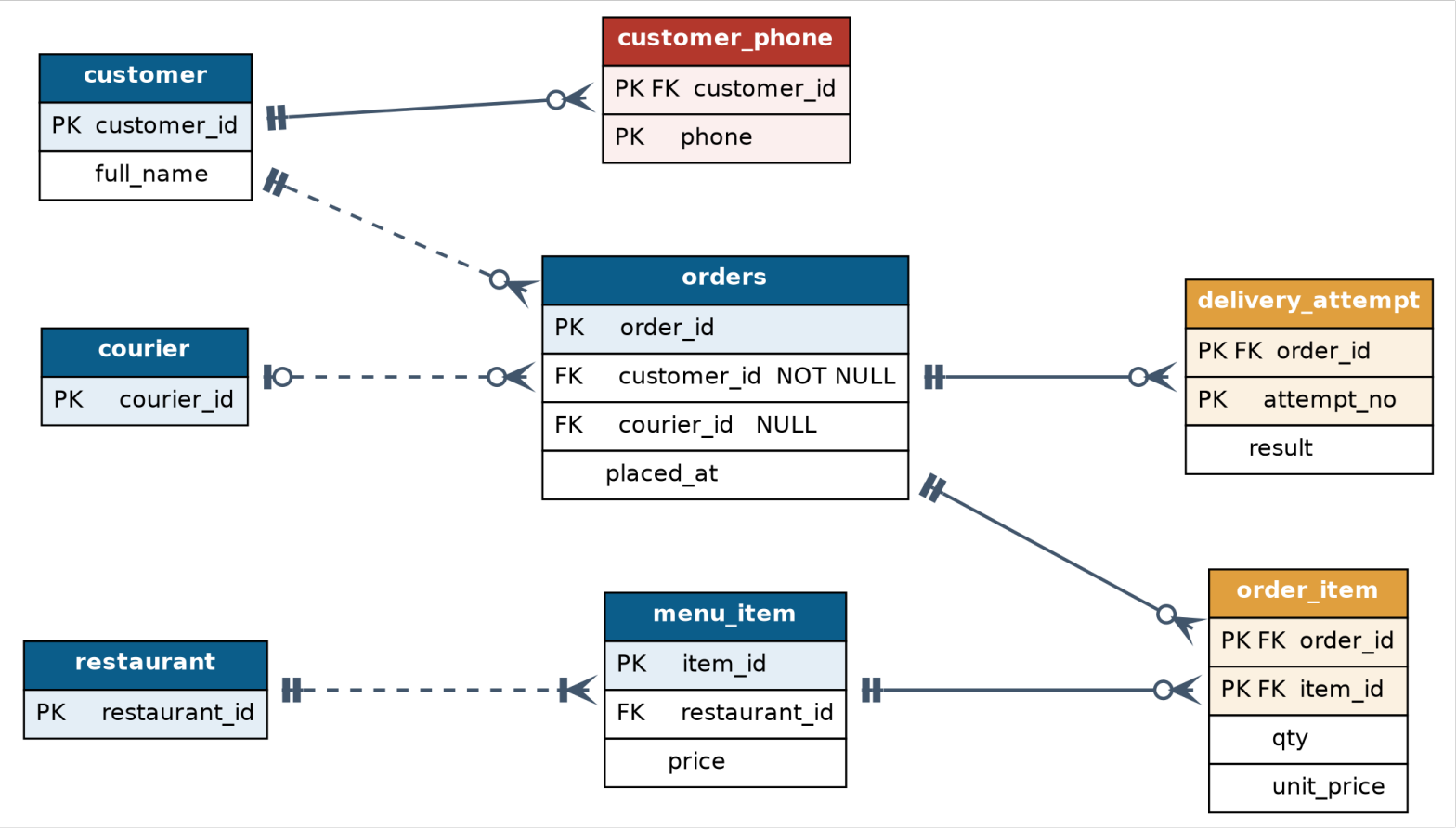
The running example as an ERD



Read every line out loud in both directions. Double line = total participation → NOT NULL. Double diamond + double rectangle = a weak entity whose PK starts with order_id.

ONE DOMAIN, TWO NOTATIONS

The same model in crow's foot



WHAT CHANGED ON THE WAY HERE

- phone was a double oval → its own table customer_phone
- contains was M:N → the table order_item with a composite PK
- DeliveryAttempt was weak → its PK starts with order_id
- order_total was dashed → it is gone; compute it
- Solid line = identifying: the parent key sits inside the child PK
- Dashed line = plain FK; courier_id is NULL-able because participation was partial

3

Mapping ERD → relations

Seven rules. Applied in order, they leave nothing to taste.

CHEAT SHEET

The seven mapping rules

#	ER construct	Relational result
1	Strong entity	A table. Its attributes become columns.
2	Key attribute	PRIMARY KEY of that table.
3	Composite attribute	One column per component — drop the container name.
4	Multivalued attribute	A new table (owner_pk, value); PK = both columns together.
5	1 : N relationship	FK on the N side, pointing at the 1 side. No new table.
6	M : N relationship	A new table with FKs to both sides; PK = the two FKs (+ discriminator if needed).
7	Weak entity	A table with FK to the owner; PK = owner PK + partial key; ON DELETE CASCADE.

Bonus rule for 1 : 1 — put the FK on the side with TOTAL participation and add a UNIQUE constraint on it. If both sides are total, ask whether they are really two entities at all.

1 : N

Rule 5 in detail — the FK goes on the "many" side

WRONG — A LIST INSIDE THE 1 SIDE

```
restaurant(restaurant_id, name,
           item_ids '17,18,42')
```

- Breaks 1NF — the cell is not atomic
- No FK constraint is possible: item 42 may not exist
- "Which restaurant sells item 18?" needs a string scan

RIGHT — AN FK ON THE N SIDE

```
menu_item(item_id PK,
          restaurant_id FK NOT NULL,
          name, current_price)
```

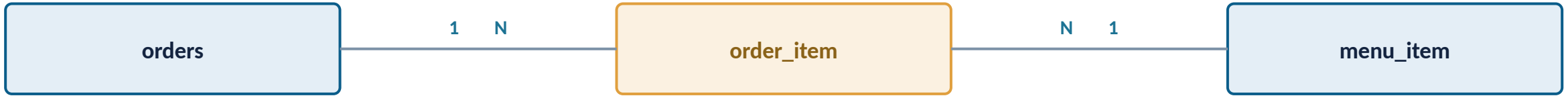
- One value per cell, enforced by the database
- NOT NULL encodes total participation
- Both directions are one join away

```
CREATE TABLE menu_item (
  item_id        bigserial PRIMARY KEY,
  restaurant_id  bigint NOT NULL REFERENCES restaurant(restaurant_id) ON DELETE CASCADE,
  name           text   NOT NULL,
  current_price  numeric(10,2) NOT NULL CHECK (current_price >= 0),
  UNIQUE (restaurant_id, name)      -- one item name per restaurant
);
```

The UNIQUE line is the business rule you would otherwise lose: the ERD said "a restaurant does not have two identical dishes".

M : N

Rule 6 in detail — M:N always becomes a table



One M:N becomes two 1:N

```
CREATE TABLE order_item (
  order_id    bigint NOT NULL REFERENCES orders(order_id)
                     ON DELETE CASCADE,
  item_id     bigint NOT NULL REFERENCES menu_item(item_id)
                     ON DELETE RESTRICT,
  qty         int    NOT NULL CHECK (qty > 0),
  unit_price  numeric(10,2) NOT NULL,   -- price CHARGED
  PRIMARY KEY (order_id, item_id)
);
```

WHY THE JUNCTION TABLE MATTERS

- It is where the interesting attributes live: qty, unit_price, discount
- Composite PK blocks the same item twice in one order
- Two different ON DELETE actions — deliberate, not accidental

THE MOST IMPORTANT LINE ON THIS SLIDE

unit_price is stored, not joined from menu_item. Prices change; an order must remember what the customer actually paid. This is not redundancy — it is a historical fact.

1:1 · MULTIVALUED · WEAK ENTITY

The remaining three cases

1 : 1

Courier ↔ Vehicle

```
courier(courier_id PK, …)
vehicle(vehicle_id PK,
        courier_id UNIQUE
                   REFERENCES courier)
```

FK goes on the side that cannot exist alone; UNIQUE turns 1:N into 1:1.

Multivalued

customer phones

```
customer_phone(
  customer_id FK,
  phone,
  PRIMARY KEY (customer_id, phone))
```

The whole row is the key. Add a phone_type column if the business distinguishes them.

Weak entity

delivery attempts

```
delivery_attempt(
  order_id FK,
  attempt_no,
  attempted_at, result,
  PRIMARY KEY (order_id, attempt_no))
```

Partial key attempt_no is unique only inside its order — 1, 2, 3 repeat across orders.

10 MINUTES, AT THE WHITEBOARD

Practice 2 — model it, then map it

KBTU LAB BOOKING SYSTEM — THE RULES AS THE CLIENT STATED THEM

- A lab contains many workstations; a workstation belongs to exactly one lab.
- A student may book a workstation for a time slot. A booking has a status (planned / used / cancelled).
- A student may have many bookings; a workstation may have many bookings.
- Two students must never hold the same workstation for the same slot.
- Every workstation has a set of installed software packages; the same package is installed on many workstations.

a. Draw the ERD: entities, relationships, cardinalities, total/partial participation.

b. Map it with the seven rules. Write every PRIMARY KEY and FOREIGN KEY explicitly.

c. Name the two M:N relationships and the tables they produce.

d. Which single constraint encodes rule 4? Write it as SQL.

Full solution in the speaker notes. Let two pairs put their ERDs side by side before revealing it — the differences are the lesson.

4

Normalization: 1NF → BCNF

Every normal form removes one specific way a table can lie to you.

THE MOTIVATION

Why normalize? Because unnormalized tables lie

delivery_log — one row per order, exactly as exported from the manager's Excel file

order_id	customer	restaurant	restaurant_address	items	courier
4711	Aigul S.	Sandyq	Tole bi 59	Beshbarmak x1, Cola x2	Marat
4712	Daniyar K.	Sandyq	Tole bi 59	Lagman x1	Marat
4713	Aigul S.	Del Papa	Abay 22	Pizza x1, Salad x1	Alina

Insert anomaly

A new restaurant signs the contract on Monday. You cannot store its address until somebody orders from it — there is no row to put it in.

Update anomaly

Sandyq moves to a new address. You must find and change every row that mentions it. Miss one and the table now holds two different truths.

Delete anomaly

Cancel order 4713 and Del Papa's address disappears from the database entirely, along with the fact that the restaurant exists.

All three anomalies have ONE cause: this table stores facts about four different things — orders, customers, restaurants and couriers — in one row.

X → Y

Functional dependencies: the tool for the whole part

`X → Y  reads: "if two rows agree on X, they MUST agree on Y"`

X is called the determinant. Y is functionally dependent on X.

FDS IN DELIVERY_LOG

```
order_id        →  customer, restaurant, courier
restaurant      →  restaurant_address
customer_id     →  customer_name, phone
{order_id, item_id} →  qty, unit_price
```

Every arrow that is not "key → rest" is a candidate for a new table.

THE MISTAKE TO AVOID

FDs come from business rules, not from the rows you can see.

In today's data every customer has a distinct phone number. That does NOT make phone → customer_id a dependency — it makes it a coincidence that the next INSERT can destroy.

Working method: for every attribute ask "what is the SMALLEST set of columns I must know to determine it?" — the answer is its determinant.

FIRST NORMAL FORM

1NF — one atomic value per cell

Rule: every attribute holds a single, indivisible value; no repeating groups; every row is identified by the key.

BEFORE — A LIST IN A CELL

order_id	items
4711	Beshbarmak x1, Cola x2
4712	Lagman x1



AFTER — ONE ROW PER VALUE

order_id	item_name	qty
4711	Beshbarmak	1
4711	Cola	2
4712	Lagman	1

THE OTHER 1NF SMELL

phone1, phone2, phone3 — a repeating group spread across columns. Symptom: the fourth phone needs an ALTER TABLE.

POSTGRESQL CAVEAT

arrays and jsonb let you break 1NF legally. Use them for data you never filter or join on — not as a shortcut around modelling.

SECOND NORMAL FORM

2NF — no partial dependency on a composite key

Rule: in 1NF, and no non-key attribute depends on only PART of a composite primary key. (If the PK is a single column, 2NF is automatic.)

VIOLATION

```
order_item ( order_id, item_id, qty, item_name, item_price )
PRIMARY KEY (order_id, item_id)

{order_id, item_id}  →  qty          full dependency
item_id              →  item_name     PARTIAL
item_id              →  item_price    PARTIAL
```

Smell: the dish name is repeated in every single order line that contains it. Rename the dish and you rewrite thousands of rows.

FIX — SPLIT OFF WHAT DEPENDS ON PART OF THE KEY

```
order_item(order_id, item_id, qty, unit_price)
  PRIMARY KEY (order_id, item_id)

menu_item(item_id PK, name, current_price)
```

- item_name now stored exactly once
- unit_price stays in order_item — it depends on the whole key (this order, this item)
- current_price lives in menu_item — a different fact

2NF only ever has something to say about tables with a composite primary key. Surrogate-key tables skip straight to 3NF.

THIRD NORMAL FORM

3NF — no non-key attribute depends on another

Rule: in 2NF, and no transitive dependency — no non-key attribute is determined by another non-key attribute.

VIOLATION

```
orders ( order_id PK, placed_at,
         restaurant_id, restaurant_address )

order_id       →  restaurant_id
restaurant_id  →  restaurant_address
──────────────────────────────────
order_id       →  restaurant_address   (transitive)
```

Smell: "if I know the restaurant, I already know the address" — and the address is repeated in every order.

FIX — THE INTERMEDIATE DETERMINANT BECOMES A TABLE

```
orders(order_id PK, placed_at,
       restaurant_id FK -> restaurant)

restaurant(restaurant_id PK, name, address)
```

- The address is stored once and changes once
- A restaurant can now exist before its first order — the insert anomaly is gone
- Deleting the last order no longer erases the restaurant

Mnemonic for 1NF–3NF: every non-key attribute must depend on the key, the whole key, and nothing but the key.

BOYCE-CODD NORMAL FORM

BCNF — every determinant must be a superkey

Rule: for every non-trivial FD X → Y, X must be a superkey. Stricter than 3NF, which forgives the case where Y is part of a key.

IN 3NF BUT NOT IN BCNF

```
tutoring ( student_id, subject, tutor_id )
```

Rules: a tutor teaches exactly ONE subject;
a student has one tutor per subject.

```
{student_id, subject} → tutor_id
tutor_id → subject
```

Candidate keys: {student_id, subject} and {student_id, tutor_id}.
tutor_id is a determinant but NOT a superkey → BCNF violated.
subject is part of a key, so 3NF is still satisfied.

DECOMPOSITION — AND ITS PRICE

```
tutor ( tutor_id PK, subject )
student_tutor ( student_id, tutor_id,
                PRIMARY KEY (student_id, tutor_id) )
```

- Redundancy gone: a tutor's subject is stored once
- But the FD {student_id, subject} → tutor_id now spans two tables and cannot be enforced by a single constraint
- BCNF is always achievable; dependency preservation is not

In practice: aim for 3NF everywhere, go to BCNF when overlapping candidate keys cause real redundancy — and document the FD you gave up.

REFERENCE

The four forms on one page

Form	Requirement	Smell that gives it away	Fix
1NF	Atomic values, no repeating groups	Comma-separated cells; columns phone1, phone2, phone3	One row per value; new child table
2NF	1NF + no partial dependency on a composite key	A column that only depends on half of the key; a name repeated on every line	Move that column to a table keyed by that half
3NF	2NF + no transitive dependency	"If I know X I already know Y" between two non-key columns	Turn the intermediate determinant into its own table
BCNF	Every determinant is a superkey	Overlapping candidate keys; a non-key column determining part of a key	Decompose; accept possible loss of dependency preservation

"The key, the whole key, and nothing but the key — so help me Codd." *1NF+2NF: the whole key · 3NF: nothing but the key · BCNF: nothing but a key*

Practical target for coursework and for production OLTP schemas: 3NF, with BCNF where it costs nothing.

WORKED EXAMPLE

Putting it together: delivery_log, step by step

0NF

`delivery_log(order_id, customer, restaurant, restaurant_address, items, courier)`

One row per order, items in a list. All three anomalies present.

1NF

`order_line(order_id, item_name, qty, …) — one row per item`

Atomic cells. New composite key (order_id, item_name).

2NF

`split off menu_item(item_id, name, current_price)`

item_name and price depended on item_id only — half of the key.

3NF

`split off customer, restaurant, courier`

restaurant → address and customer → phone were transitive.

> Each step was driven by a functional dependency, not by intuition. Ask for the FD before allowing the split.

WORKED EXAMPLE

The result: six tables, each about exactly one thing

customer

```
customer_id PK
full_name
phone
```

restaurant

```
restaurant_id PK
name
address
```

courier

```
courier_id PK
full_name
phone
```

menu_item

```
item_id PK
restaurant_id FK
name
current_price
```

orders

```
order_id PK
customer_id FK
restaurant_id FK
courier_id FK NULL
placed_at, status
```

order_item

```
order_id FK  ┐ PK
item_id  FK  ┘
qty
unit_price
```

> Test every table with one sentence: "this table stores facts about ___". If you need the word "and", it is not normalized yet.

REALITY CHECK

When to break the rules on purpose

LEGITIMATE REASONS TO DENORMALIZE

- Historical values: unit_price on the order line — the fact itself is different
- Reporting tables and dashboards where reads vastly outnumber writes
- Precomputed aggregates you have measured with EXPLAIN ANALYZE and cannot afford to compute live
- In PostgreSQL, prefer a materialized view over a hand-maintained duplicate column

BAD REASONS YOU WILL HEAR

- "Joins are slow" — with a proper index on the FK, usually they are not
- "It is easier to write the query" — that is what views are for
- "We might need it later" — you will need consistency sooner
- "The ORM generates it that way" — the ORM is not the designer

The working rule

Normalize to 3NF first. Denormalize only against evidence — a measured query plan — and write down in a comment which rule you broke and why.

TAKE THIS AWAY

Five things to remember

1. A key is a claim about every possible state of the table — not about today's rows.
2. Cardinality decides the number of tables. M:N always adds one you did not draw.
3. Mapping an ERD is mechanical. If you are improvising, the model is unfinished.
4. Every normal form removes one named anomaly. Name the anomaly, then split.
5. Normalize to 3NF by default; break it only with measurements and a comment.

Next week: SQL DDL and constraints — turning this schema into a database that refuses bad data.